Algoritmos Distribuidos
Transferencia de Datos en Ambiente Fiable

Andrés Sanoja

Claudia León

Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

Octubre 2008 – Marzo 2022



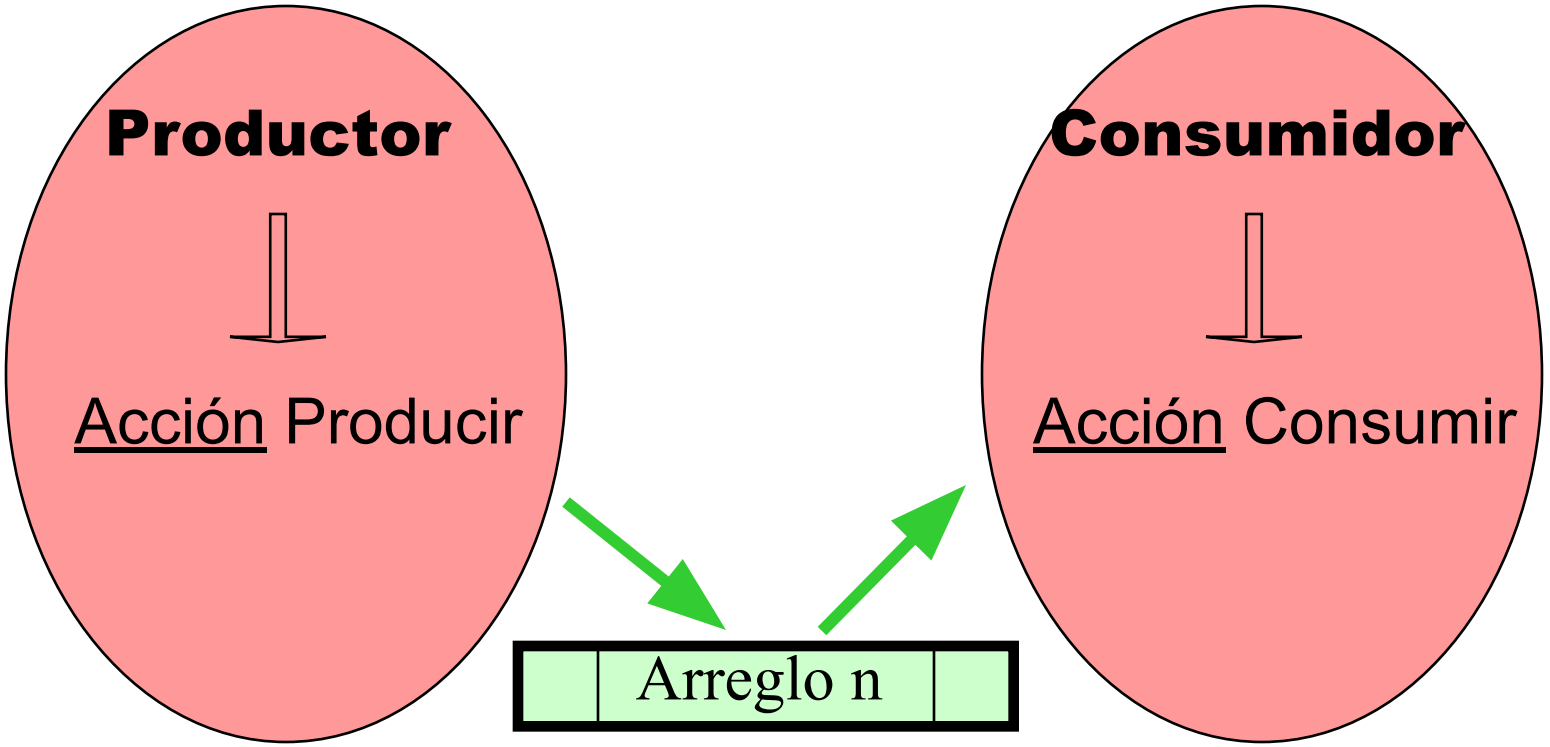
Objetivos

- Familiarizarnos con una notación para expresar algoritmos distribuidos.
- Establecer diferencias entre algoritmos centralizados y algoritmos distribuidos.
- Estudiar soluciones al problema de transferencia de información entre dos procesos, en ambientes fiables

Transferencia de datos

- Una entidad llamada Productor envía mensajes a otra llamada Consumidor
- Los mensajes producidos y aún no consumidos deben ser memorizados hasta su consumo
- Una vez emitido, un mensaje debe ser consumido una sóla vez
- Suponemos una capacidad de almacenamiento de n mensajes

Productor Consumidor



Especificación de restricciones

Sean las operaciones ***Producir y Consumir*** que permiten el envío y recepción de mensajes. Estas operaciones deben ser sincronizadas para garantizar que cada mensaje producido sea consumido una y sólo una vez.

Consideremos los siguientes **contadores** asociados a cada operación **Oper:**

- #aut(Oper) : Cantidad de ejecuciones autorizadas
- #term(Oper): Cantidad de ejecuciones terminadas

Estos contadores, inicializados en 0, son:

- Monótonos crecientes

∀∀ Oper: #aut(Oper) ≥ #term(Oper)

- La diferencia #aut(Oper) - #term(Oper) representa la cantidad de ejecuciones en curso de Oper.

Especificación de restricciones

- #term(Producir): Cantidad de mensajes emitidos
- #term(Consumir): Cantidad de mensajes consumidos

Un envío es posible sólo si el productor no tiene un avance de n mensajes y un consumo es posible sólo si hay mensajes producidos y aún no consumidos:

Condición(Producir): #aut(Producir) < #term(Consumir) + n
Condición(Consumir): #aut(Consumir) < #term(Producir)

Es decir:

Condición(Producir):#aut(Producir) - #term(Consumir) < n
Condición(Consumir):#aut(Consumir) - #term(Producir) < 0

Especificación de restricciones

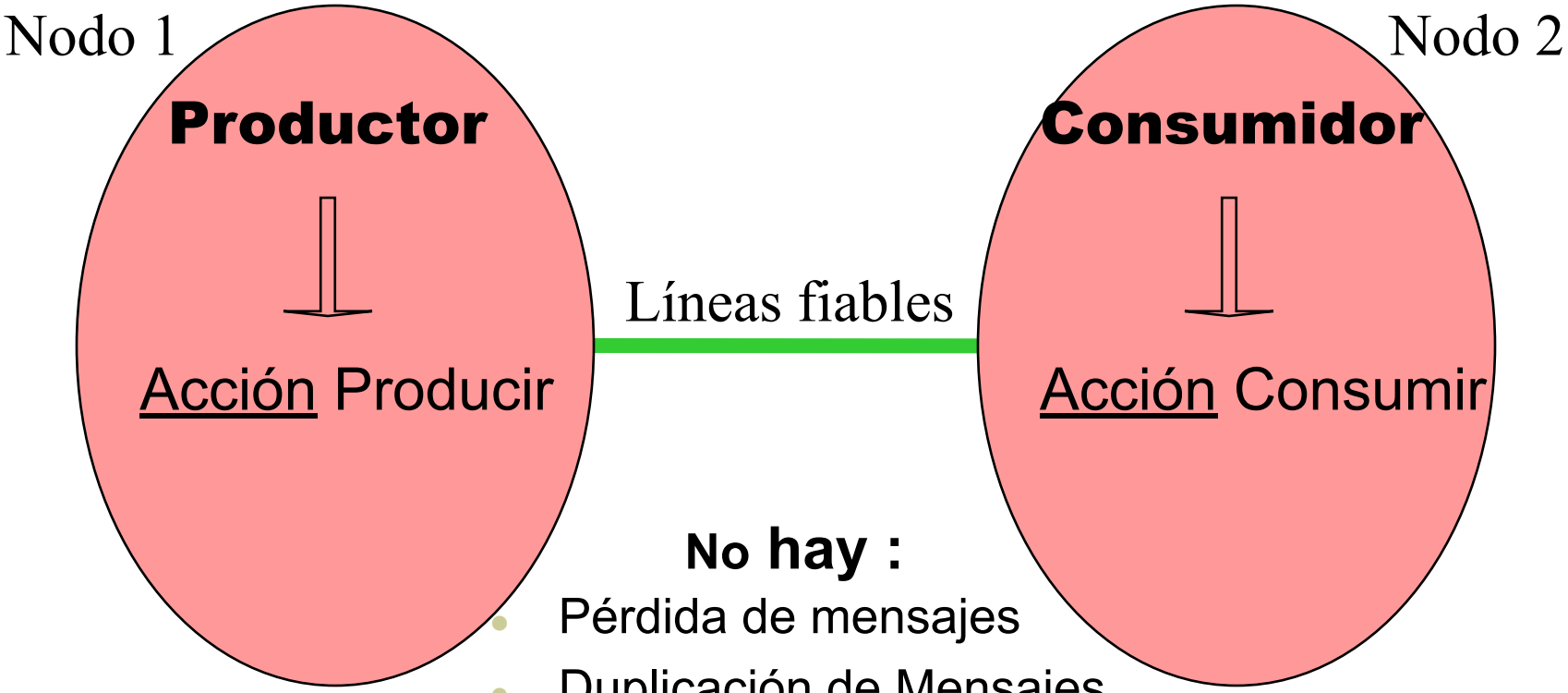
Cuando una llamada a la operación Oper es autorizada, se incrementa automáticamente el contador: #aut(Oper)

Cuando una llamada a la operación Oper finaliza, se incrementa automáticamente el contador: #term(Oper)

Solución Centralizada

```
Monitor ProductorConsumidor
Var buffer : arreglo [0 .. n-1] de mensajes;
       ent,sal: o .. N-1 inicializa 0; señal p,c.
       autPro,termPro,autCon,termCon: entero inicializa 0;
Accion Producir(m: mensaje)
    Si autPro - termCon ≥ n entonces esperar(p) fsi;
    autPro := autPro + 1;
    buffer[ent]:=m; ent:= (ent + 1) mod n;
    termPro := termPro + 1;
    Si autCon - termPro < 0 entonces señalar(c) fsi;
faccion Producir;
Accion Consumir(m: mensaje)
    Si autCon - termPro ≥ 0 entonces esperar(c) fsi;
    autCon := autCon + 1;
    m :=buffer[sal]; sal:= (sal + 1) mod n;
    termCon := termCon + 1;
    Si autPro - termCon < n entonces señalar(p) fsi;
faccion Consumir;
fmonitor;
```

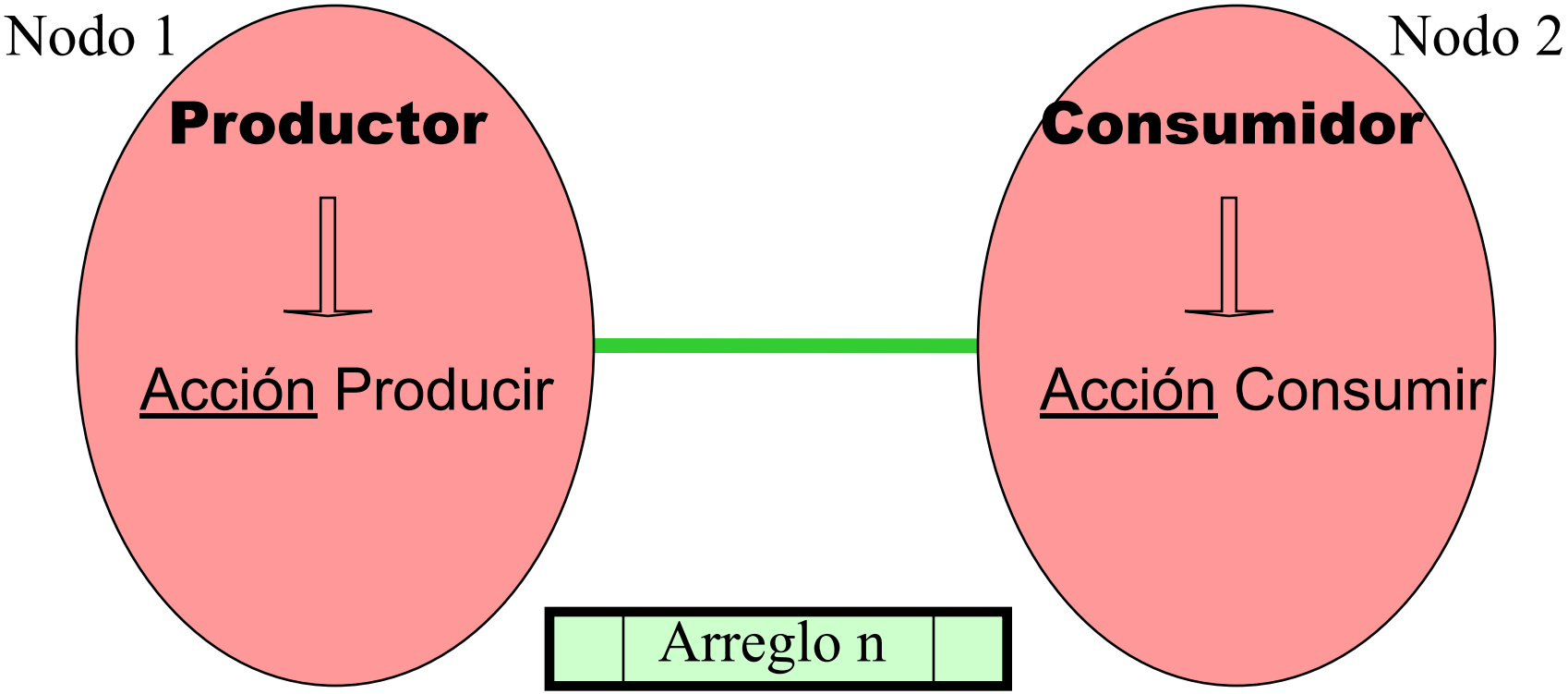
Solución distribuida



No hay :

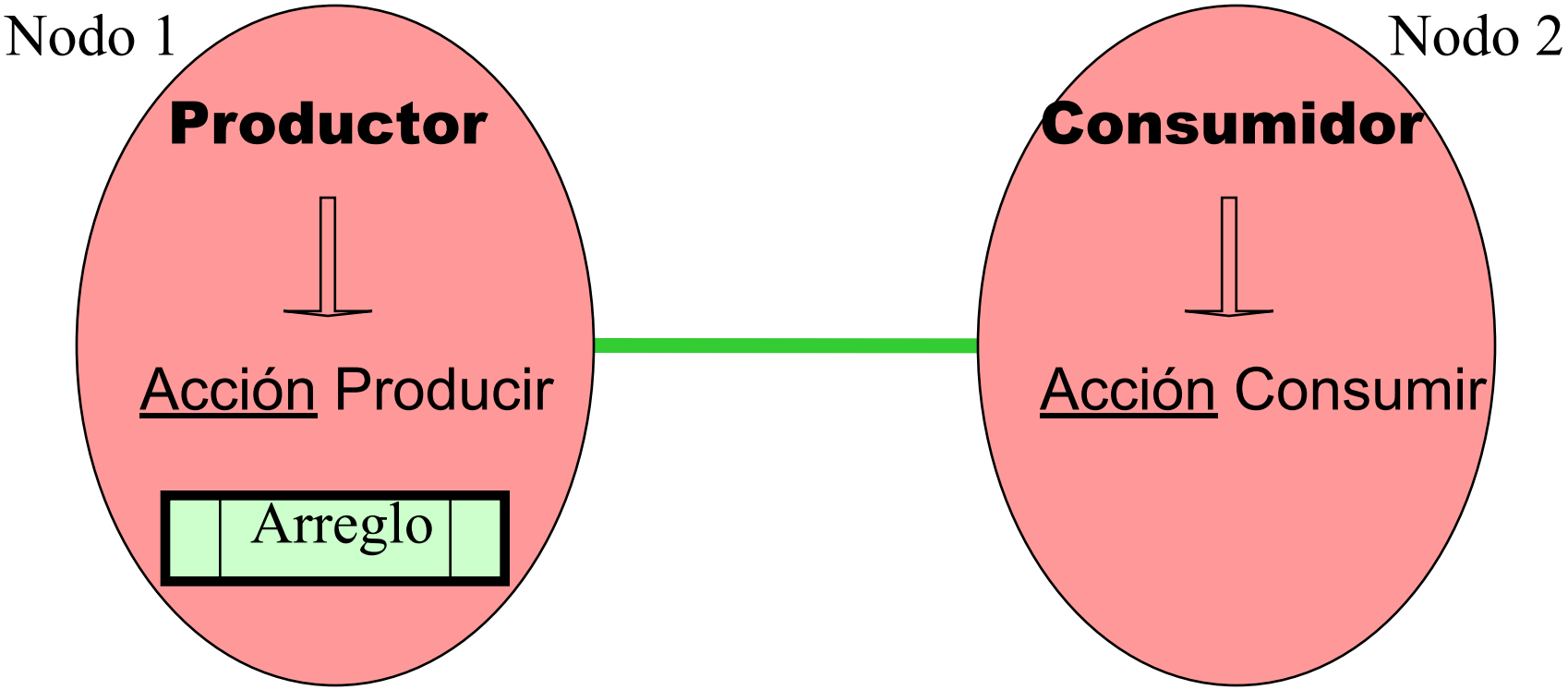
- Pérdida de mensajes
- Duplicación de Mensajes
- Alteración de Mensajes
- Pérdida de secuencia

Solución distribuida

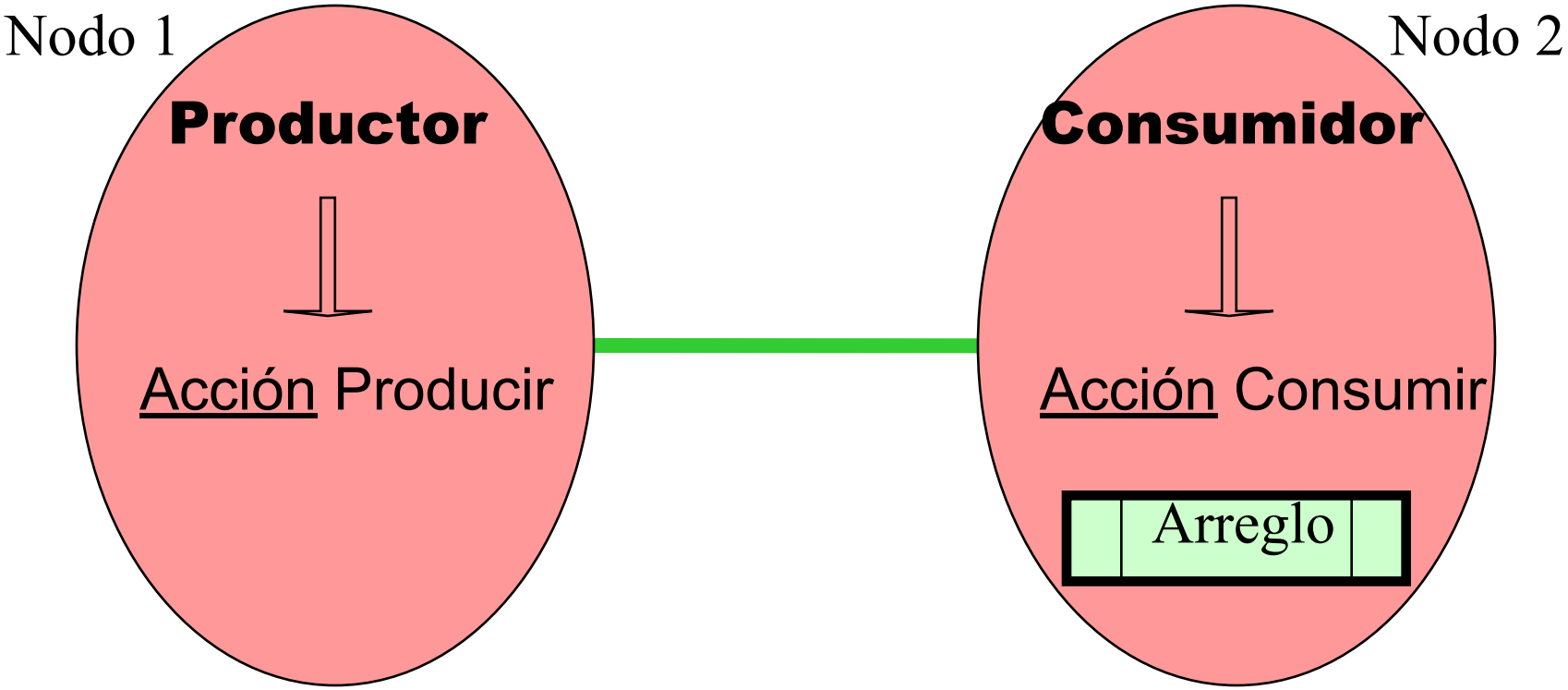


Donde se almacena el arreglo ?

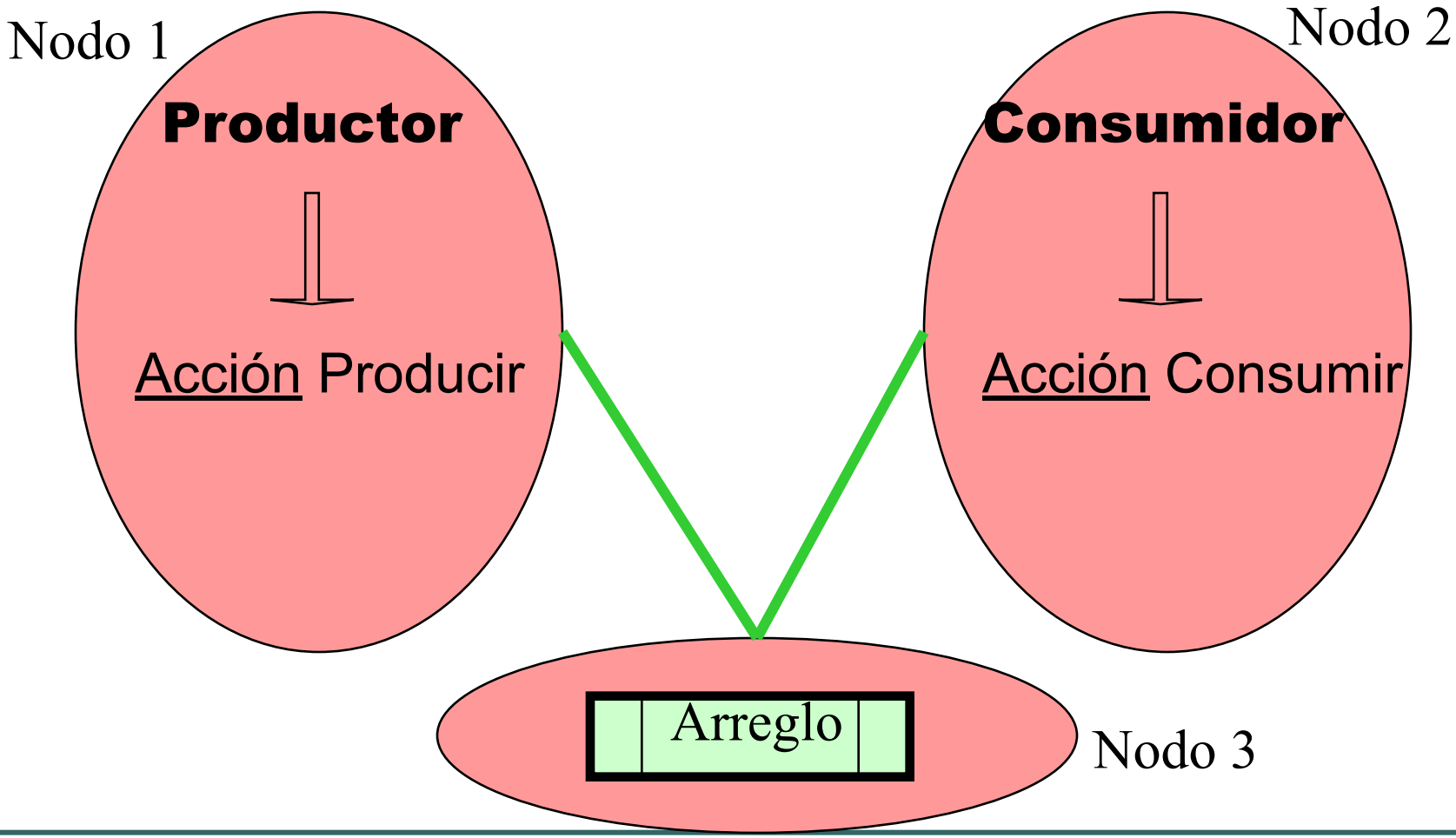
Opción 1: En el productor



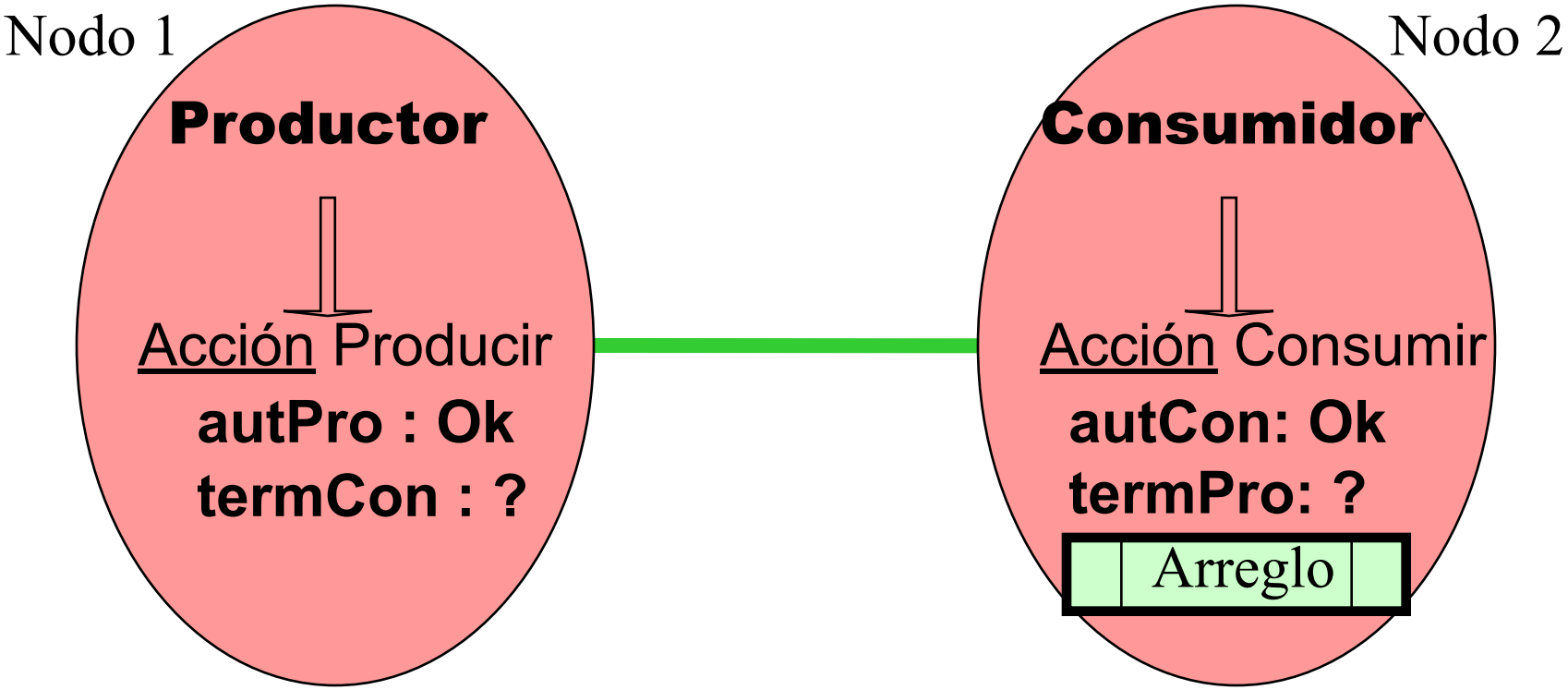
Opción 2: En el consumidor



Opción 3: En otro nodo



Solución distribuida: Opción 2



Condición(Producir): autPro - termCon < n
Condición(Consumir): autCon - termPro < 0

Eventos: Su localización y su observación

Los nodos deben intercambiar información de control para evaluar correctamente las condiciones de Producir y Consumir

Problema fundamental en los A.D.:

Se debe distinguir entre el nodo donde se produce un evento, y por lo tanto su observación corresponde al valor exacto de una variable, y el nodo sobre el cual el valor de esa variable va a ser utilizado

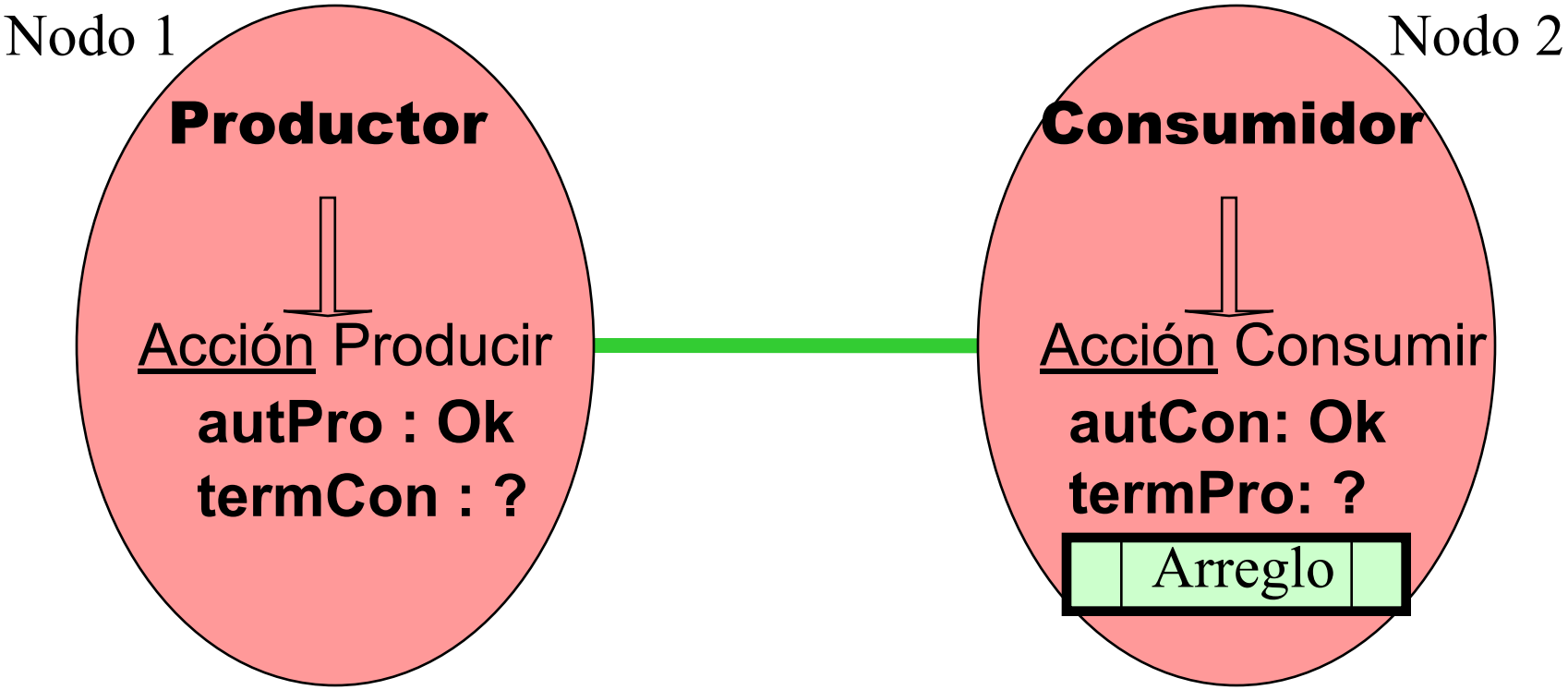
Solución:

- Colocar una copia de la variable en cada nodo que la utilice
- Introducir reglas o protocolos que garanticen que las copias evolucionan igual que la variable “original”.

Protocolos para actualización de copias

1. Un nodo que utiliza una copia de una variable solicita el valor al nodo que maneja dicha variable cada vez que deba evaluar una expresión en la cual esa variable interfiere
2. El nodo que maneja una variable difunde sistemáticamente las modificaciones que realiza sobre ella a todos los nodos que posean copia de dicha variable.

Solución distribuida: Opción 2



Condición(Producir): autPro - termCon < n
Condición(Consumir): autCon - termPro < 0

Nodo del Consumidor

```
Var buffer : arreglo [0 .. n-1]  de mensaje;
       ent,sal: o .. N-1 inicializa 0;
       termPro,autCon : entero inicializa 0;

Cuando  se invoque a Consumir(m: mensaje) hacer
Esperar (autCon - termPro < 0)
    autCon := autCon + 1;
    m :=buffer[sal]; sal:= (sal + 1) mod n;
    Enviar (Incremento,*) a Productor;
fesperar;
fcuando

Cuando  llegue (m: mensaje) hacer
    Recibir (*,m) de Productor;
    buffer[ent]:=m; ent:= (ent + 1) mod n;
    termPro := termPro + 1;
fcuando
```

Nodo del Productor

```
Var autPro,CtermCon : entero inicializa 0;

Cuando  se invoque a Producir(m: mensaje) hacer
Esperar (autPro - CtermCon < n)
    autPro := autPro + 1;
    Enviar (*,m) a Consumidor;
fesperar;
fcuando

Cuando  llegue  (Incremento,*)
    Recibir (Incremento,*) de Consumidor;
    CtermCon := CtermCon + 1;
fcuando
```

Debido a que los contadores X son monótonos crecientes:

∀ Oper, X: Copiade X(Oper) ≤ X (Oper)

CtermCon ≤ termCon

Características de los algoritmos distribuidos resaltadas en esta aplicación

- Ausencia de un estado global que un proceso pueda captar instantáneamente
- Cada nodo mantiene las variables que le conciernen
- El tiempo de transmisión de un mensaje no es nulo
- Un proceso no puede saber si el valor de su copia es exacto al de la variable o uno de sus valores anteriores
- Si los contadores son monótonos creciente se puede sustituir una condición abstracta por una concreta más fuerte.

